UNIT 5 – GRAPHS

Topic: Graph Operations Using Adjacency Matrix and Adjacency List

1. Problem Statement

Develop a menu-driven C++ program for graph operations, including vertex and edge insertion, deletion, graph display, BFS, and DFS. Provide options to display the graph as an adjacency matrix or adjacency list, update the graph, and show the result after each operation.

2. Objectives

1. To understand graph representation using an adjacency matrix and adjacency list.
2. To perform vertex and edge insertion and deletion.
3. To implement Breadth-First Search (BFS) and Depth-First Search (DFS).
4. To display and update graph information.
5. To understand graph traversal using C++.

3. Software Requirements

- **Programming Language:** C++
- **IDE:** Code::Blocks
- **Operating System:** Windows
- **Header Files:** iostream, vector, queue

4. Theory

A graph is a non-linear data structure consisting of vertices and edges. Vertices represent objects, while edges represent connections between them.

Graph Representation

1. Adjacency Matrix

An adjacency matrix is a two-dimensional array used to represent connections between vertices. A value of 1 indicates an edge, while 0 indicates no edge.

2. Adjacency List

An adjacency list stores the neighbouring vertices of each vertex. It is useful for representing sparse graphs.

Graph Traversal

Breadth-First Search (BFS): BFS visits neighbouring vertices first and uses a queue to manage traversal.

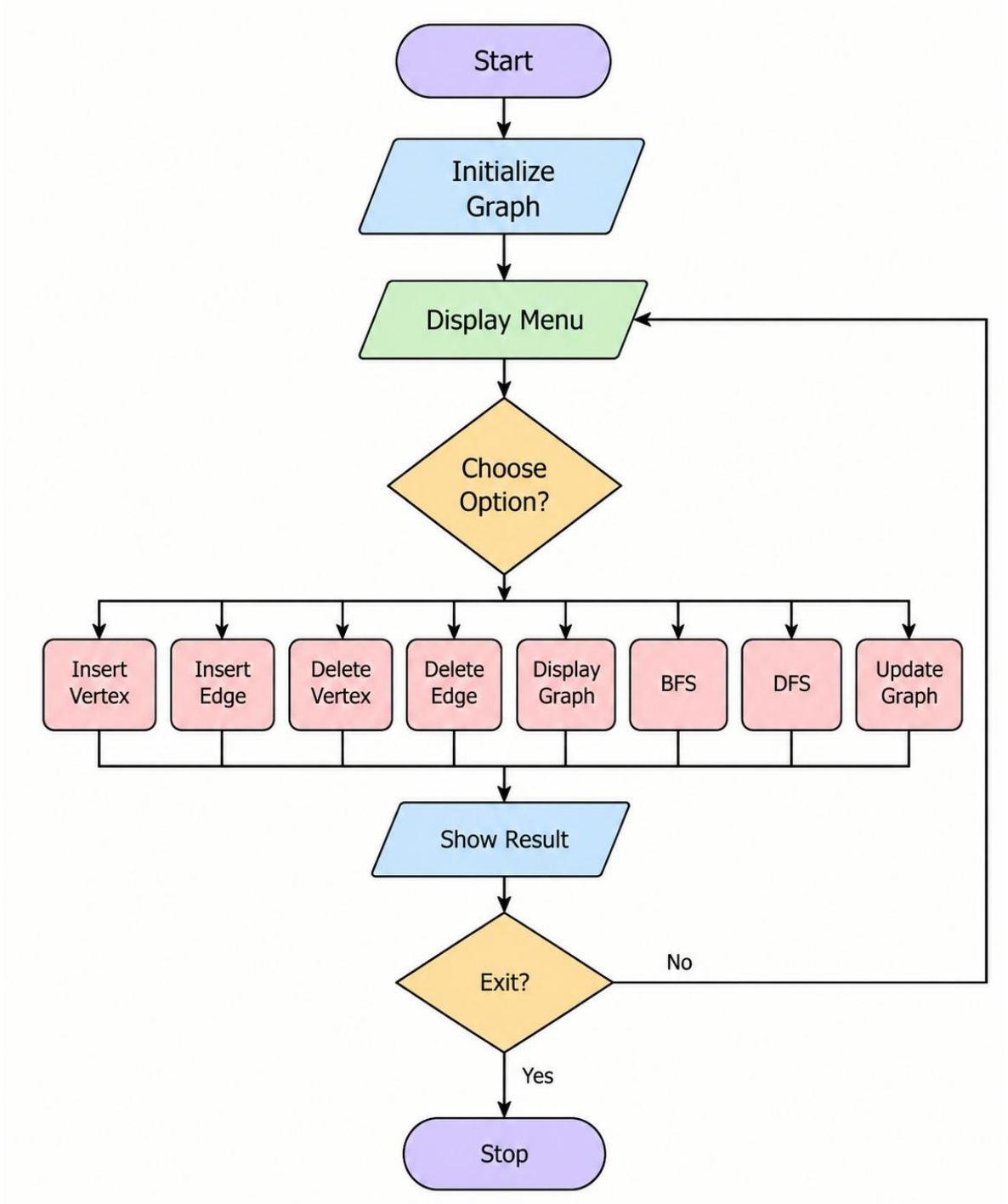
Depth-First Search (DFS): DFS explores one path as deeply as possible before returning to explore other paths. It can be implemented using recursion or a stack.

5. Algorithm

1. Start the program.
2. Initialize an empty graph.
3. Display the menu of graph operations.

4. Accept the user's choice.
5. Perform the selected operation:
    - Insert a vertex.
    - Insert an edge.
    - Delete a vertex.
    - Delete an edge.
    - Display the adjacency matrix.
    - Display the adjacency list.
    - Perform BFS traversal.
    - Perform DFS traversal.
    - Update the graph.
6. Display the operation result.
7. Repeat the menu until the user selects Exit.
8. Stop the program.

6. Flowchart



7. Source Code

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

vector<int> adj[100];
int vertices = 0;

void insertVertex() {
    if (vertices >= 100) {
        cout << "Maximum vertices reached.\n";
        return;
    }

    vertices++;
    cout << "Vertex " << vertices - 1
         << " added successfully.\n";
}

void insertEdge() {
    int u, v;

    cout << "Enter two vertex numbers: ";
    cin >> u >> v;

    if (u < 0 || v < 0 || u >= vertices || v >= vertices) {
        cout << "Invalid vertex number.\n";
        return;
    }

    int i;

    for (i = 0; i < (int)adj[u].size(); i++) {
        if (adj[u][i] == v) {
            cout << "Edge already exists.\n";
            return;
        }
    }

    adj[u].push_back(v);
    adj[v].push_back(u);

    cout << "Edge added successfully.\n";
}

void deleteEdge() {
    int u, v, i;

    cout << "Enter edge vertices to delete: ";
    cin >> u >> v;

    if (u < 0 || v < 0 || u >= vertices || v >= vertices) {
        cout << "Invalid vertex number.\n";
        return;
    }

    for (i = 0; i < (int)adj[u].size(); i++) {
        if (adj[u][i] == v) {
            adj[u].erase(adj[u].begin() + i);
            break;
        }
    }

    for (i = 0; i < (int)adj[v].size(); i++) {
        if (adj[v][i] == u) {
            adj[v].erase(adj[v].begin() + i);
```

```
            break;
        }
    }

    cout << "Edge deleted if it existed.\n";
}

void deleteVertex() {
    int v, i, j;

    cout << "Enter vertex number to delete: ";
    cin >> v;

    if (v < 0 || v >= vertices) {
        cout << "Invalid vertex number.\n";
        return;
    }

    for (i = 0; i < vertices; i++) {
        for (j = 0; j < (int)adj[i].size(); j++) {
            if (adj[i][j] == v) {
                adj[i].erase(adj[i].begin() + j);
                j--;
            } else if (adj[i][j] > v) {
                adj[i][j]--;
            }
        }
    }

    for (i = v; i < vertices - 1; i++) {
        adj[i] = adj[i + 1];
    }

    adj[vertices - 1].clear();
    vertices--;

    cout << "Vertex deleted successfully.\n";
}

void displayMatrix() {
    int i, j, k, found;

    cout << "\nAdjacency Matrix:\n";

    for (i = 0; i < vertices; i++) {
        for (j = 0; j < vertices; j++) {
            found = 0;

            for (k = 0; k < (int)adj[i].size(); k++) {
                if (adj[i][k] == j) {
                    found = 1;
                    break;
                }
            }

            cout << found << " ";
        }
        cout << endl;
    }
}

void displayList() {
    int i, j;

    cout << "\nAdjacency List:\n";

    for (i = 0; i < vertices; i++) {
        cout << i << " -> ";
```

```
        for (j = 0; j < (int)adj[i].size(); j++) {
            cout << adj[i][j] << " ";
        }

        cout << endl;
    }
}

void BFS() {
    int start, v, i;

    cout << "Enter starting vertex: ";
    cin >> start;

    if (start < 0 || start >= vertices) {
        cout << "Invalid starting vertex.\n";
        return;
    }
    vector<bool> visited(vertices, false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    cout << "BFS Traversal: ";

    while (!q.empty()) {
        v = q.front();
        q.pop();

        cout << v << " ";

        for (i = 0; i < (int)adj[v].size(); i++) {
            int x = adj[v][i];

            if (!visited[x]) {
                visited[x] = true;
                q.push(x);
            }
        }
    }

    cout << endl;
}

void DFSUtil(int v, vector<bool>& visited) {
    int i, x;

    visited[v] = true;
    cout << v << " ";

    for (i = 0; i < (int)adj[v].size(); i++) {
        x = adj[v][i];

        if (!visited[x]) {
            DFSUtil(x, visited);
        }
    }
}
void DFS() {
    int start;

    cout << "Enter starting vertex: ";
    cin >> start;

    if (start < 0 || start >= vertices) {
        cout << "Invalid starting vertex.\n";
```

```
        return;
    }

    vector<bool> visited(vertices, false);

    cout << "DFS Traversal: ";
    DFSUtil(start, visited);
    cout << endl;
}

void updateGraph() {
    int choice;
    cout << "\nUpdate Graph:\n";
    cout << "1. Insert Vertex\n";
    cout << "2. Insert Edge\n";
    cout << "3. Delete Vertex\n";
    cout << "4. Delete Edge\n";
    cout << "Enter choice: ";
    cin >> choice;

    switch (choice) {
        case 1: insertVertex(); break;
        case 2: insertEdge(); break;
        case 3: deleteVertex(); break;
        case 4: deleteEdge(); break;
        default: cout << "Invalid choice.\n";
    }

    displayList();
}

int main() {
    int choice;

    do {
        cout << "\n===== GRAPH OPERATIONS =====\n";
        cout << "1. Insert Vertex\n";
        cout << "2. Insert Edge\n";
        cout << "3. Delete Vertex\n";
        cout << "4. Delete Edge\n";
        cout << "5. Display Adjacency Matrix\n";
        cout << "6. Display Adjacency List\n";
        cout << "7. BFS Traversal\n";
        cout << "8. DFS Traversal\n";
        cout << "9. Update Graph\n";
        cout << "10. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1: insertVertex(); break;
            case 2: insertEdge(); break;
            case 3: deleteVertex(); break;
            case 4: deleteEdge(); break;
            case 5: displayMatrix(); break;
            case 6: displayList(); break;
            case 7: BFS(); break;
            case 8: DFS(); break;
            case 9: updateGraph(); break;
            case 10: cout << "Program ended.\n"; break;
            default: cout << "Invalid choice.\n";
        }

    } while (choice != 10);

    return 0;
}
```

8. Sample Output

```
===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 1
Vertex 0 added successfully.

===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 1
Vertex 1 added successfully.

===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice:
1
Vertex 2 added successfully.
```

```
===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 2
Enter two vertex numbers: 0 1
Edge added successfully.

===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 2
Enter two vertex numbers: 1 2
Edge added successfully.

===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 6

Adjacency List:
0 -> 1
1 -> 0 2
2 -> 1
```

```
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 5

Adjacency Matrix:
0 1 0
1 0 1
0 1 0

===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 7
Enter starting vertex: 0
BFS Traversal: 0 1 2

===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 8
Enter starting vertex: 0
DFS Traversal: 0 1 2

===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 4
Enter edge vertices to delete: 1 2
Edge deleted if it existed.

===== GRAPH OPERATIONS =====
1. Insert Vertex
2. Insert Edge
3. Delete Vertex
4. Delete Edge
5. Display Adjacency Matrix
6. Display Adjacency List
7. BFS Traversal
8. DFS Traversal
9. Update Graph
10. Exit
Enter choice: 6

Adjacency List:
0 -> 1
1 -> 0
2 ->
```

9. Applications

1. Campus navigation systems.
2. Social networking connections.
3. Computer network representation.
4. Route and path exploration.
5. Network connectivity analysis.

10. Result

The menu-driven C++ program was developed to perform basic graph operations. It supports insertion and deletion of vertices and edges, graph representation using an adjacency matrix and adjacency list, and graph traversal using BFS and DFS.

11. Conclusion

This practical helped in understanding graph data structures and their representations. It also provided practical knowledge of BFS and DFS traversal techniques using C++.